

WORKSHEET 01

Python fundamentals, under interview pressure

15 medium-hard questions

Predict output, diagnose bugs and implement small components. The questions are original practice exercises based on the topics in your supplied preparation notes.

How to use this worksheet

- Attempt output questions before running the code. Include every printed line and explain the cause, not just the result.
- For implementation questions, write runnable Python in your editor. Use the page space for your approach, edge cases and complexity.
- Treat each question as independent, starting in a fresh interpreter. Snippets assume Python 3.11 or later; run asyncio examples as scripts.
- Work in three sessions: Q1-5, Q6-10 and Q11-15. Use the time targets as practice goals. Open the separate answer key after each session.

Question map

01-02 Collections; recursive traversal

03-04 Mutability; copying; closure capture

05-06 Exceptions; validation; batch error isolation

07-08 Dataclasses; equality; mutable fields; hashing

09-10 Generators; send/close; streaming sliding window

11-12 Parameterized decorators; wrapping order

13-15 Coroutines; concurrency; async context and iteration

Name: _____ Date: _____

04 Two hidden shared-state bugs

Mutability + closures • Hard • Target: 9 minutes

1. Predict both print statements. Explain the shared default argument and the loop-variable capture separately.
2. Fix `make_record` so each call gets a fresh history by default, while allowing a caller to supply a shared list explicitly. Fix the lambdas so their factors are 0, 1 and 2.
3. Show the output after your fixes using the same calls. Assume each version runs in a fresh interpreter.

```
def make_record(value, history=[]):
    history.append(value)
    return history

first = make_record("A")
second = make_record("B")
print(first, second, first is second)

multipliers = []
for factor in range(3):
    multipliers.append(lambda n: n * factor)

print([fn(10) for fn in multipliers])
```

Your answer / reasoning / test cases

10 Streaming sliding-window sums

Generators + sliding window • Hard • Target: 15 minutes

1. Implement `moving_sum(values, width)`, a lazy generator of sums for every complete consecutive window. `values` is a one-pass iterable of integers, possibly infinite.
2. `width` must be a positive built-in int; reject bool and invalid widths with `ValueError` on first advancement. An input shorter than `width` produces no output.
3. Consume each input once. Use $O(\text{width})$ auxiliary memory and $O(n)$ time for n consumed items, assuming constant-time arithmetic. Do not materialize the iterable or recompute each full window sum.
4. Explain why invalid-width validation inside this generator occurs on advancement rather than when it is called.

```
def moving_sum(values, width):
    # Implement this as a generator.
    ...

# Required behavior:
# list(moving_sum(iter([4, -1, 2, 7]), 3)) -> [5, 8]
# list(moving_sum([5, 6], 1))             -> [5, 6]
# list(moving_sum([1, 2], 3))             -> []
# next(moving_sum([], 0))                  -> ValueError
```

Your answer / reasoning / test cases

11 A parameterized role decorator

Decorators • Medium • Target: 12 minutes

1. Implement `require_role(*allowed_roles)` for synchronous functions whose first argument is a user dict. The optional role value is a string. Missing or disallowed roles raise `PermissionError` before the function executes, an empty `allowed-role` list denies all users.
2. Forward the remaining positional and keyword arguments, return the original result, and preserve `__name__`, `__doc__` and `__wrapped__` using `functools.wraps`.
3. Test an allowed role, a denied role and a missing role. Prove the wrapped function is not called on a denied request.

```
@require_role("admin", "auditor")
def export_report(user, report_id, *, format="csv"):
    """Export one report."""
    return f"{report_id}:{format}"

# Required behavior:
# export_report({"role": "admin"}, 42, format="json")
#     -> "42:json"
# export_report({"role": "reader"}, 42)
#     -> PermissionError
# export_report({}, 42) -> PermissionError
# export_report.__name__ == "export_report"
```

Your answer / reasoning / test cases

12 Decorator order changes exception handling

Decorators + exceptions • Hard • Target: 10 minutes

1. Write the complete output, including the A and B result lines. Express each decorated function as a composition of trace and fallback.
2. Would fallback handle `int(None)`? Explain which exception is raised and what trace still prints.

```
from functools import wraps

def trace(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("enter")
        try:
            return func(*args, **kwargs)
        finally:
            print("exit")
    return wrapper

def fallback(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        try:
            return func(*args, **kwargs)
        except ValueError:
            print("fallback")
            return -1
    return wrapper

@fallback
@trace
def parse_a(value):
    return int(value)

@trace
@fallback
def parse_b(value):
    return int(value)

print("A", parse_a("bad"))
print("B", parse_b("bad"))
```

Your answer / reasoning / test cases

15 Async iteration with reliable context cleanup

async with + async for • Hard • Target: 15 minutes

1. Complete `Feed.__aenter__` and `Feed.__aexit__`. Entering prints open and returns the feed; exiting must await `aclose` and must not suppress exceptions.
2. Implement `async first_two(source)` using `async with` and `async for`. Return at most two values. Stop early once two are read, while still closing the feed. Propagate a `ValueError` from iteration.
3. Give the output/exception order for `["A", "B", "C"]`, `[]`, and `["A", "ERROR", "C"]`. Explain what returning `True` from `__aexit__` would change in the error case.

```
class Feed:
    def __init__(self, values):
        self._values = iter(values)
    async def __aenter__(self):
        print("open")
        # TODO
    async def __aexit__(self, exc_type, exc, tb):
        # TODO
        pass
    def __aiter__(self):
        return self
    async def __anext__(self):
        try:
            value = next(self._values)
        except StopIteration:
            raise StopAsyncIteration
        if value == "ERROR":
            raise ValueError("bad item")
        return value
    async def aclose(self):
        print("close")

# Implement first_two(source).
```

Your answer / reasoning / test cases
